

Process Scheduling

- When a computer is multiprogrammed, it frequently has multiple processes or threads competing for the CPU at the same time. This situation occurs whenever two or more of them are simultaneously in the ready state.
- If only one CPU is available, a choice has to be made which process to run next. The part of the operating system that makes the choice is called the **scheduler**, and the algorithm it uses is called the **scheduling algorithm**.

*The **process scheduling** is the activity of the process scheduler that handles the removal of the running process from the CPU and the selection of another process on the basis of a particular strategy.*

☑ **Process scheduler** selects among available processes for next execution on CPU core

Who cares about scheduling algorithms?

- ☞ Batch servers
- ☞ Time sharing machines
- ☞ Networked servers
- ☞ You care if you have a bunch of users and/or if the demands of the jobs differ

Who doesn't care about scheduling algorithms?

- ☞ PC's
 - One user who only competes with himself for the CPU

Process Behavior

Nearly all *processes alternate bursts of computing with (disk or network) I/O requests*. (Process execution consists of cycle of CPU execution and I/O wait).

Some processes, such as the one in following figure (a), *spend most of their time computing*, while other processes, such as the one shown in (b), *spend most of their time waiting for I/O*.

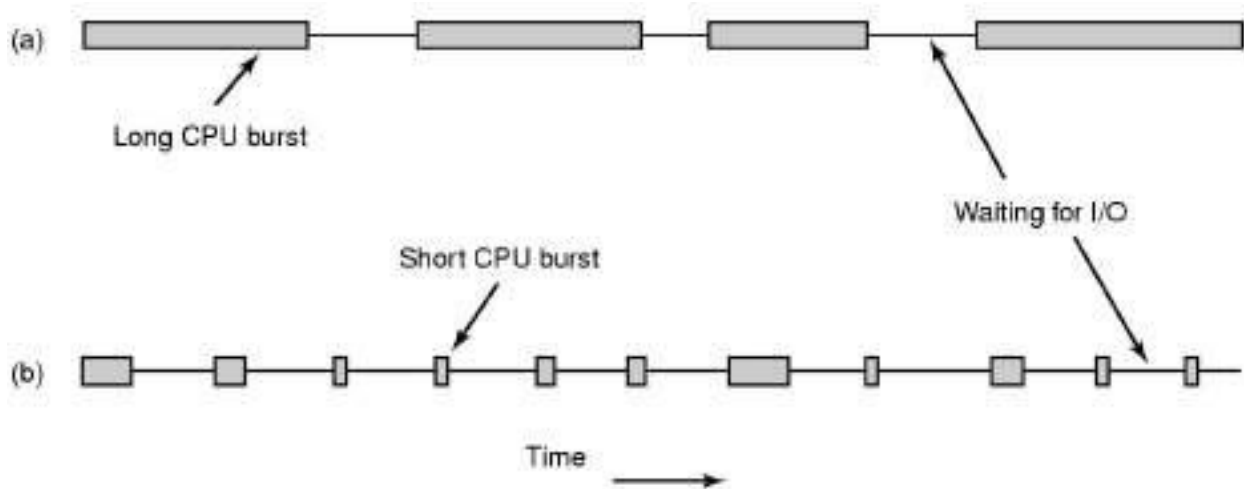


Figure: Bursts of CPU usage alternate with periods of waiting for I/O. (a) A CPU-bound process. (b) An I/O-bound process

- ☑ The processes which spend most of their time computing are called **compute-bound or CPU bound process**
- ☑ The processes which spend most of their time waiting for I/O are called **I/O bound process**
- ☑ **CPU Burst** is the time that a process spends on the CPU executing some code. CPU Burst deals with the **running state** of the process.
- ☑ **I/O Burst** is the time that a process spends in waiting for the completion of the I/O request. I/O Burst deals with the **waiting state** of the process.
- ☑ *Compute-bound processes typically have long CPU bursts and thus infrequent I/O waits, whereas I/O-bound processes have short CPU bursts and thus frequent I/O waits.*
- ☑ Note that the **key factor is the length of the CPU burst, not the length of the I/O burst.**
 - I/O-bound processes are I/O bound because they do not compute much between I/O requests, not because they have especially long I/O requests.
 - It takes the same time to issue the hardware request to read a disk block no matter how much or how little time it takes to process the data after they arrive.

The basic idea: *If an I/O-bound process wants to run, it should get a chance quickly so that it can issue its disk request and keep the disk busy*

CPU Scheduler and Dispatcher

- ☑ Whenever the CPU becomes idle, the operating system must select one of the processes in the ready queue to be executed. The selection process is carried out by the **short-term scheduler** (or **CPU scheduler**). The scheduler selects a process from the processes in memory that are ready to execute and allocates the CPU to that processes.
- ☑ The **dispatcher** is the module that gives control of the CPU to the processes selected by the short-term scheduler. The time it takes for the dispatcher to stop one process and start another running is known as the **dispatch latency**.

When to Schedule?

A key issue related to scheduling is when to make scheduling decisions. It turns out that there are a variety of situations in which scheduling is needed.

- *When a new process is created*, a decision needs to be made whether to run the parent process or the child process.
- *When a process exits*. The process can no longer run (since it no longer exists), so some other process must be chosen from the set of ready processes.
- *When a process blocks on I/O*, on a semaphore, or for some other reason, another process has to be selected to run.
- *When an I/O interrupt occurs*. If the interrupt came from an I/O device that has now completed its work, some process that was blocked waiting for the I/O may now be ready to run. It is up to the scheduler to decide whether to run the newly ready process, the process that was running at the time of the interrupt, or some third process.

Preemptive and Nonpreemptive Scheduling

Scheduling algorithms can be divided into two categories with respect to how they deal with clock interrupts.

Nonpreemptive Scheduling

- A **nonpreemptive** scheduling algorithm picks a process to run and then just lets it run until it blocks (either on I/O or waiting for another process) or voluntarily releases the CPU
- Even if it runs for many hours, it will not be forcibly suspended. In effect, no scheduling decisions are made during clock interrupts.

Preemptive Scheduling

- A **preemptive** scheduling algorithm picks a process and lets it run for a maximum of some fixed time.
- If it is still running at the end of the time interval, it is suspended and the scheduler picks another process to run (if one is available).
- Doing preemptive scheduling requires having a clock interrupt occur at the end of the time interval to give control of the CPU back to the scheduler.

In preemptive scheduling, the CPU is allocated to the processes for a limited time whereas, in Non-preemptive scheduling, the CPU is allocated to the process till it terminates or switches to the waiting state.

Scheduling Criteria

Different CPU-scheduling algorithms have different properties, and the choice of a particular algorithm may favor one class of processes over another. In choosing which algorithm to use in a particular situation, we must consider the properties of the various algorithms. Many criteria have been suggested for comparing CPU-scheduling algorithms. Some of them are as follows:

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – number of processes that complete their execution per time unit
- **Turnaround time** – amount of time to execute a particular process
- **Waiting time** – amount of time a process has been waiting in the ready queue
- **Response time** – amount of time it takes from when a request was submitted until the first response is produced

Turnaround Time = Completion Time – Arrival Time

Waiting Time = Turnaround Time – Burst Time

Response Time = First CPU Entry Time – Arrival Time

Categories of Scheduling Algorithms

In different environments different scheduling algorithms are needed. This situation arises because different application areas (and different kinds of operating systems) have different goals. In other words, what the scheduler should optimize for is not the same in all systems. Three environments worth distinguishing are

1. Batch systems.
2. Interactive systems.
3. Real time systems.

Scheduling Algorithm Goals

In order to design a scheduling algorithm, it is necessary to have some idea of what a good algorithm should do. Some goals depend on the environment (batch, interactive, or real time), but some are desirable in all cases. Some goals are listed below:

All systems

Fairness - giving each process a fair share of the CPU
Policy enforcement - seeing that stated policy is carried out
Balance - keeping all parts of the system busy

Batch systems

Throughput - maximize jobs per hour
Turnaround time - minimize time between submission and termination
CPU utilization - keep the CPU busy all the time

Interactive systems

Response time - respond to requests quickly
Proportionality - meet users' expectations

Real-time systems

Meeting deadlines - avoid losing data
Predictability - avoid quality degradation in multimedia systems

Scheduling Algorithm Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time

- Min waiting time
- Min response time

Batch System Scheduling

- ❖ First-come first-served
- ❖ Shortest job first
- ❖ Shortest remaining time next

First- Come, First-Served (FCFS) Scheduling

- ❖ By far the simplest CPU-scheduling algorithm is the **first-come first-serve (FCFS)** scheduling algorithm.
- ❖ With this scheme, *the process that requests the CPU first is allocated the CPU first.*
- ❖ Processes are scheduled in the order they arrived.
- ❖ The implementation of the FCFS policy is easily managed with a FIFO queue.
- ❖ When a process enters the ready queue, its PCB is linked onto the tail of the queue. When the CPU is free, it is allocated to the process at the head of the queue. The running process is then removed from the queue.
- ❖ FCFS scheduling algorithm is nonpreemptive
- ❖ Example:

Process ID	Arrival Time	Burst Time
P1	4	5
P2	6	4
P3	0	3
P4	6	2
P5	5	4

Gantt Chart for FCFS scheduling:



We know,

$$\text{Turnaround Time} = \text{Completion Time} - \text{Arrival Time}$$

$$\text{Waiting Time} = \text{Turnaround Time} - \text{Burst Time}$$

Process ID	Completion Time	Turnaround Time	Waiting Time
P1	9	$9 - 4 = 5$	$5 - 5 = 0$
P2	17	$17 - 6 = 11$	$11 - 4 = 7$
P3	3	$3 - 0 = 3$	$3 - 3 = 0$
P4	19	$19 - 6 = 13$	$13 - 2 = 11$
P5	13	$13 - 5 = 8$	$8 - 4 = 4$

— Average Turnaround Time = $(5+11+3+13+8)/5 = 8$ unit

— Average Turnaround Time = $(0+7+0+11+4)/5 = 4.4$ unit

Advantage:

- ☞ It is simple and fair.
- ☞ It can be easily implemented using queue data structure.
- ☞ It does not lead to starvation since every process gets chance to execute

Disadvantages:

- ☞ The average waiting time under the FCFS policy is often quite long.
- ☞ It does not consider the priority or burst time of the processes.
- ☞ Convoy effect is possible.
 - All small I/O bound processes wait for one big CPU bound process to acquire CPU.
 - Short process behind long process

Problem-01:

Consider the set of 5 processes whose arrival time and burst time are given below. If the CPU scheduling policy is FCFS, calculate the average waiting time and average turn-around time.

Process	Arrival time	Burst time
P1	3	4
P2	5	3
P3	0	2
P4	5	1
P5	4	3

Solution:

Gantt Chart-



Here, black box represents the idle time of CPU.

Now, we know-

- Turnaround Time = Completion Time – Arrival Time
- Waiting time = Turnaround Time – Burst time

Process	Completion time	Turnaround Time	Waiting Time
P1	7	$7 - 3 = 4$	$4 - 4 = 0$
P2	13	$13 - 5 = 8$	$8 - 3 = 5$
P3	2	$2 - 0 = 2$	$2 - 2 = 0$
P4	14	$14 - 5 = 9$	$9 - 1 = 8$
P5	10	$10 - 4 = 6$	$6 - 3 = 3$

$$\text{Average Turnaround Time} = (4 + 8 + 2 + 9 + 6) / 5 = 29 / 5 = 5.8 \text{ unit}$$

$$\text{Average Waiting Time} = (0 + 5 + 0 + 8 + 3) / 5 = 16 / 5 = 3.2 \text{ unit}$$

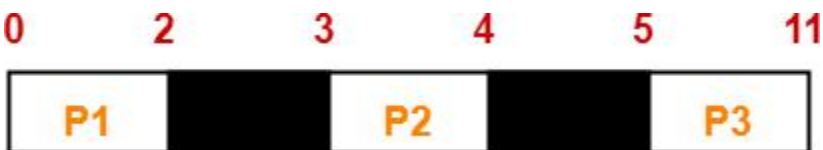
Problem-02:

Consider the set of 3 processes whose arrival time and burst time are given below. If the CPU scheduling policy is FCFS, calculate the average waiting time and average turn-around time.

Process	Arrival Time	Burst Time
P1	0	2
P2	3	1
P3	5	6

Solution:

Gantt Chart-



Here, black box represents the idle time of CPU.

Now, we know-

- Turnaround Time = Completion Time – Arrival Time
- Waiting time = Turnaround Time – Burst time

Process	Completion Time	Turnaround time	Waiting time
P1	2	$2 - 0 = 2$	$2 - 2 = 0$
P2	4	$4 - 3 = 1$	$1 - 1 = 0$
P3	11	$11 - 5 = 6$	$6 - 6 = 0$

Now,

$$\text{Average Turnaround time} = (2 + 1 + 6) / 3 = 9 / 3 = 3 \text{ unit}$$

$$\text{Average waiting time} = (0 + 0 + 0) / 3 = 0 / 3 = 0 \text{ unit}$$

Shortest-Job-First (SJF) Scheduling

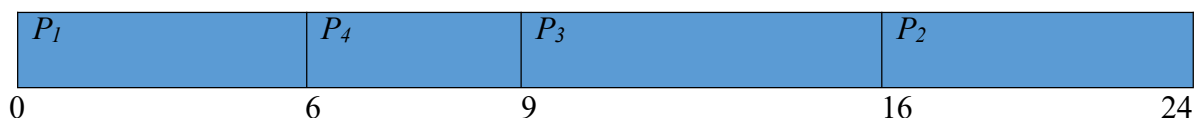
- Need to know run times in advance
- The process, that requires shortest time to complete execution is served first
- Associate with each process the length of its next CPU burst
 - Use these lengths to schedule the process with the shortest time
- The implementation of the SJF strategy is easily managed by using a FIFO queue.
 - All processes in a queue are sorted in ascending order by their required CPU bursts
 - When CPU becomes free, a process from first position in a queue is selected to run
- SJF may be optimal – gives minimum average waiting time for a given set of processes.
(Optimal only when the jobs are available simultaneously)

- Types: Preemptive and Nonpreemptive

Example:

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	6
P_2	2.0	8
P_3	4.0	7
P_4	5.0	3

- SJF (preemptive) scheduling chart:



- Turnaround Time = Completion Time – Arrival Time
- Waiting time = Turnaround Time – Burst time

Process	Completion Time	Burst Time	Turnaround Time	Waiting Time
P_1	6	6	$6 - 0 = 6$	$6 - 6 = 0$
P_2	24	8	$24 - 2 = 22$	$22 - 8 = 14$
P_3	16	7	$16 - 4 = 12$	$12 - 7 = 5$
P_4	9	3	$9 - 5 = 4$	$4 - 3 = 1$

Average Turnaround time = $(6 + 22 + 12 + 4) / 4 = 44 / 4 = 11$ unit

Average waiting time = $(0 + 14 + 5 + 1) / 3 = 20 / 4 = 5$ unit

Advantages:

- Reduced average waiting time compared to FCFS
- Good responses for short processes

Disadvantages:

- It is difficult to estimate the time required for a process to execute
- Starvation is possible for long processes. They may wait forever.

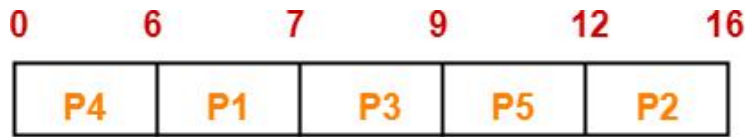
Problem-01:

Consider the set of 5 processes whose arrival time and burst time are given below. If the CPU scheduling policy is SJF non-preemptive, calculate the average waiting time and average turn - around time.

Process	Arrival Time	Burst time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

Solution-

Gantt Chart-



Now, we know-

- Turnaround time = Completion Time – Arrival time
- Waiting time = Turnaround time – Burst time

Process Id	Completion Time	Turnaround time	Waiting time
P1	7	$7 - 3 = 4$	$4 - 1 = 3$
P2	16	$16 - 1 = 15$	$15 - 4 = 11$
P3	9	$9 - 4 = 5$	$5 - 2 = 3$
P4	6	$6 - 0 = 6$	$6 - 6 = 0$
P5	12	$12 - 2 = 10$	$10 - 3 = 7$

Now,

- Average Turnaround time = $(4 + 15 + 5 + 6 + 10) / 5 = 40 / 5 = 8$ unit
- Average waiting time = $(3 + 11 + 3 + 0 + 7) / 5 = 24 / 5 = 4.8$ unit

Exercise: Schedule the given processes using SJF preemptive and find average turnaround time and average waiting time

Process ID	Burst Time	Arrival Time
P0	8	5
P1	5	0
P2	9	4
P3	2	1

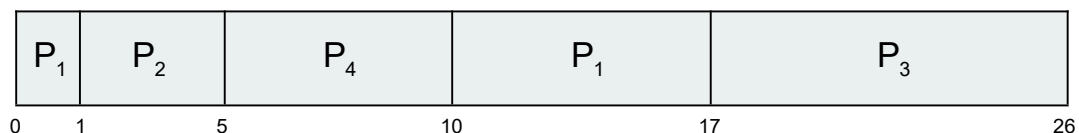
Shortest Remaining Time Next (SRTN) Scheduling

- Also called Shortest-Remaining-Time-First (SRTF) scheduling
- *The process whose remaining running time is shortest, is served first.*
- Preemptive version of SJF scheduling.
- When a new job arrives, its total time is compared to the current process' remaining time.
 - If the new job needs less time to finish than the current process, the current process is suspended and the new job is started (i.e. current process preempted)
- This strategy can also be implemented by using sorted FIFO queue.
 - All the processes in a queue are sorted in ascending order on their remaining run time
 - When CPU becomes free, a process from the first position of queue is selected to run.

Example:

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

- Preemptive SJF (or SRTN or SRTF) Gantt Chart



Process	Completion Time	Burst Time	Turnaround Time	Waiting Time
P_1	17	8	$17 - 0 = 17$	$17 - 8 = 9$
P_2	5	4	$5 - 1 = 4$	$4 - 4 = 0$
P_3	26	9	$26 - 2 = 24$	$24 - 9 = 15$
P_4	10	5	$10 - 3 = 7$	$7 - 5 = 2$

Average waiting time = $(17+4+24+7)/4 = 52/4 = 13$

Average waiting time = $(9+0+15+2)/4 = 26/4 = 6.5$

Advantages:

- Less average waiting time than SJF
- This scheme allows new short jobs to get good service.

Disadvantages:

- Requires additional computation for calculating remaining time.
- Favors short processes. Starvation is possible for long processes. Long process may wait forever.
- Context switch overhead is there.

Problem-01:

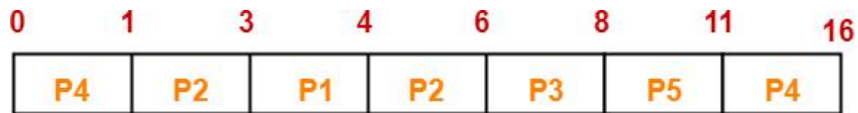
Consider the set of 5 processes whose arrival time and burst time are given below:

Process	Arrival time	Burst time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

If the CPU scheduling policy is SJF preemptive, calculate the average waiting time and average turn-around time.

Solution:

Gantt Chart:



Now, we know-

- Turnaround Time = Completion Time – Arrival Time
- Waiting time = Turnaround Time – Burst time

Process	Completion Time	Turnaround time	Waiting time
P1	4	$4 - 3 = 1$	$1 - 1 = 0$
P2	6	$6 - 1 = 5$	$5 - 4 = 1$
P3	8	$8 - 4 = 4$	$4 - 2 = 2$
P4	16	$16 - 0 = 16$	$16 - 6 = 10$
P5	11	$11 - 2 = 9$	$9 - 3 = 6$

Now,

- Average Turn Around time = $(1 + 5 + 4 + 16 + 9) / 5 = 35 / 5 = 7$ unit
- Average waiting time = $(0 + 1 + 2 + 10 + 6) / 5 = 19 / 5 = 3.8$ unit

Problem-02:

- Consider the set of 6 processes whose arrival time and burst time are given below. If the CPU scheduling policy is shortest remaining time first, calculate the average waiting time and average turn-around time.

Process	Arrival time	Burst time
P1	0	7
P2	1	5
P3	2	3
P4	3	1
P5	4	2
P6	5	1

Solution:

Gantt chart:



Process	Completion Time	Turn Around time	Waiting time
P1	19	$19 - 0 = 19$	$19 - 7 = 12$
P2	13	$13 - 1 = 12$	$12 - 5 = 7$
P3	6	$6 - 2 = 4$	$4 - 3 = 1$
P4	4	$4 - 3 = 1$	$1 - 1 = 0$
P5	9	$9 - 4 = 5$	$5 - 2 = 3$
P6	7	$7 - 5 = 2$	$2 - 1 = 1$

Average Turn Around time = $(19 + 12 + 4 + 1 + 5 + 2) / 6 = 43 / 6 = 7.17$ unit

Average waiting time = $(12 + 7 + 1 + 0 + 3 + 1) / 6 = 24 / 6 = 4$ unit

Exercises:

1. Consider the set of 3 processes whose arrival time and burst time are given below. If the CPU scheduling policy is SRTF, calculate the average waiting time and average turn-around time. (Answer : 5, 12.33)

Process	Arrival time	Burst time
P1	0	9
P2	1	4
P3	2	9

2. Consider the set of following four processes whose arrival time and burst time are given in milliseconds. Now schedule the processes using SJF and SRTF and also compare the average waiting time. (Answer: SJF = 4, SRTF = 3)

Processes	Arrival Time	Burst Time
A	0.0	7
B	2.0	4
C	4.0	1
D	5.0	4

3. Consider the set of 4 processes whose arrival time and burst time are given below. If the CPU scheduling policy is SRTF, calculate the average waiting time and average turn-around time.

Process ID	Arrival Time	Burst Time
P1	0	10
P2	3	6
P3	7	1
P4	8	3

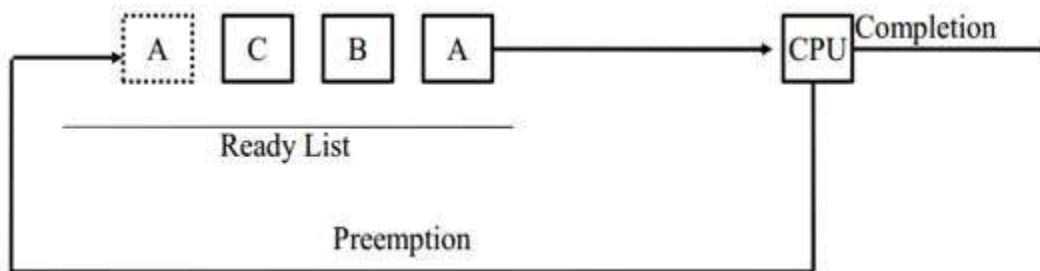


Scheduling in Interactive Systems

- ❖ Round robin
- ❖ Priority
- ❖ Multiple Queues

Round-Robin (RR) Scheduling

- ☑ One of the oldest, simplest, fairest, and most widely used algorithms
- ☑ Each process is assigned a time interval, called **time quantum or time slice**.
- ☑ The process is allowed to run only for this time interval (time quantum).
- ☑ Here, two scenarios are possible:
 - ☞ A process is either blocked or terminated before the quantum has elapsed. In this case CPU switching is done and another process (if any) is scheduled to run.
 - ☞ The process needs CPU burst longer than the time quantum. In this case process runs till the end of time quantum. Then it is preempted and another process runs. The preempted process is added to the end of ready queue.



- ☑ When a quantum time is over or process completes execution (whichever is earlier), it starts new process. Selection of new process is as per FCFS scheduling algorithm.
- ☑ This strategy can be implemented using FIFO queue.
 - ☞ If any process comes, or process releases CPU, or process is preempted. It is moved to the end of the queue.

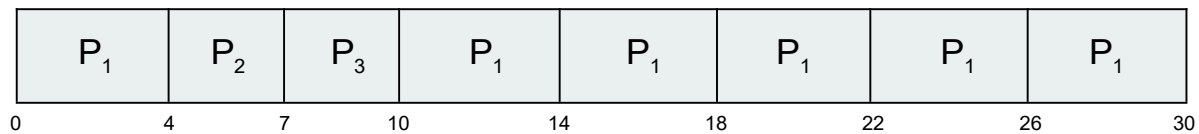
The ready queue is treated as circular queue.

- ☑ When CPU becomes free, a process from the first position in a queue is selected to run.
- ☑ If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.

Example: RR Scheduling with quantum = 4 (Arrival time is 0 for all processes)

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

The Gantt chart is:



Process ID	Completion Time	Turnaround Time	Waiting Time
P1	30	$30 - 0 = 30$	$30 - 24 = 6$
P2	7	$7 - 0 = 7$	$7 - 3 = 4$
P3	10	$10 - 0 = 10$	$10 - 3 = 7$

- Average Turnaround time = $(30 + 7 + 10) / 3 = 47 / 3 = 15.66$ unit
- Average waiting time = $(6 + 4 + 7) / 3 = 17 / 3 = 5.66$ unit

Advantages:

- ☞ It is simple, fast and most widely used algorithm.
- ☞ Don't need to know run times in advance
- ☞ It gives the best performance in terms of average response time.
- ☞ It is best suited for time sharing system, client server architecture and interactive system.

Disadvantages:

- There is the overhead of context switch.
- Its performance heavily depends on time quantum. Selection of time quantum is crucial.
 - If time quantum is too short, it causes frequent context switches and lowers CPU efficiency.
 - If it is too long, it causes poor response for short interactive processes.
- Priorities cannot be set for the processes.

NOTES:

- ☑ 80% of CPU bursts should be shorter than quantum time
- ☑ A quantum around 20–50 msec is often a reasonable compromise

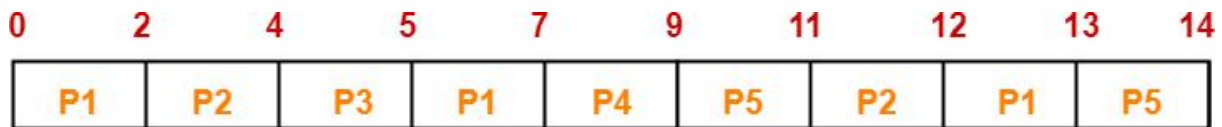
Problem-01:

Consider the set of 5 processes whose arrival time and burst time are given below. If the CPU scheduling policy is Round Robin with time quantum = 2 unit, calculate the average waiting time and average turn-around time.

Process Id	Arrival time	Burst time
P1	0	5
P2	1	3
P3	2	1
P4	3	2
P5	4	3

Solution-

Gantt Chart-



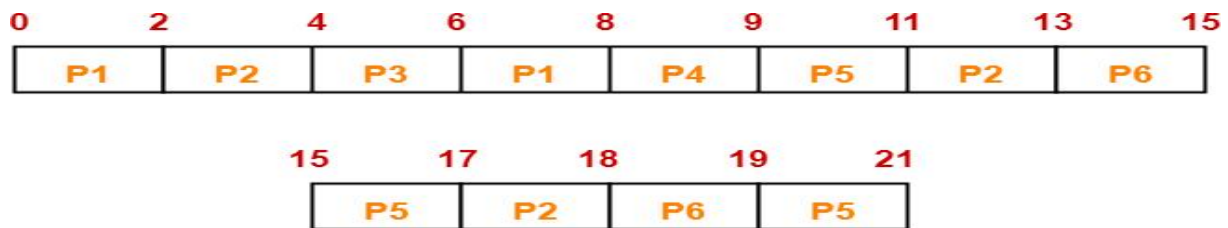
Process	Completion Time	Turn Around time	Waiting time
P1	13	$13 - 0 = 13$	$13 - 5 = 8$
P2	12	$12 - 1 = 11$	$11 - 3 = 8$
P3	5	$5 - 2 = 3$	$3 - 1 = 2$
P4	9	$9 - 3 = 6$	$6 - 2 = 4$
P5	14	$14 - 4 = 10$	$10 - 3 = 7$

- Average Turnaround time = $(13 + 11 + 3 + 6 + 10) / 5 = 43 / 5 = 8.6$ unit
- Average waiting time = $(8 + 8 + 2 + 4 + 7) / 5 = 29 / 5 = 5.8$ unit

Problem-02:

Consider the set of 6 processes whose arrival time and burst time are given below. If the CPU scheduling policy is Round Robin with time quantum = 2, calculate the average waiting time and average turn-around time.

Process	Arrival time	Burst time
P1	0	4
P2	1	5
P3	2	2
P4	3	1
P5	4	6
P6	6	3

Solution-**Gantt chart-**

Process	Completion Time	Turnaround time	Waiting time
P1	8	$8 - 0 = 8$	$8 - 4 = 4$
P2	18	$18 - 1 = 17$	$17 - 5 = 12$
P3	6	$6 - 2 = 4$	$4 - 2 = 2$
P4	9	$9 - 3 = 6$	$6 - 1 = 5$
P5	21	$21 - 4 = 17$	$17 - 6 = 11$
P6	19	$19 - 6 = 13$	$13 - 3 = 10$

- Average Turn Around time = $(8 + 17 + 4 + 6 + 17 + 13) / 6 = 65 / 6 = 10.84$ unit
- Average waiting time = $(4 + 12 + 2 + 5 + 11 + 10) / 6 = 44 / 6 = 7.33$ unit

Problem-03:

Consider the set of 4 processes whose arrival time and burst time are given below. If the CPU scheduling policy is Round Robin with time quantum = 4, calculate the average waiting time and average turn-around time.

Process	Arrival time	Burst time
P0	0	10
P1	1	6
P2	3	2
P3	5	4

Exercise:

Consider the set of 6 processes whose arrival time and burst time are given below. If the CPU scheduling policy is Round Robin with time quantum = 3, calculate the average waiting time and average turn-around time. (Answer: 16, 21.33)

Process	Arrival time	Burst time
P1	5	5
P2	4	6
P3	3	7
P4	1	9
P5	2	2
P6	6	3

Priority Scheduling

Even on a PC with a single owner, there may be multiple processes, some of them more important than others. For example, a daemon process sending electronic mail in the background should be assigned a lower priority than a process displaying a video film on the screen in real time.

IDEA: Each process is assigned a priority, and the runnable process with the highest priority is allowed to run.

- A priority is associated with each process, and the CPU is allocated to the process with the highest priority
- Priorities can be assigned to processes statically or dynamically
- Priorities are generally indicated by some fixed range of numbers, such as 0 to 10 or 0 to 4,095. However, there is no general agreement on whether 0 is the highest or lowest priority. **Some systems use low numbers to represent low priority; others use low numbers for high priority.**
- Equal-priority processes are scheduled in FCFS order.
- Round-robin scheduling makes the implicit assumption that all processes are equally important
- An SJF algorithm is simply a priority algorithm where the priority (p) is the inverse of the (predicted) next CPU burst. The larger the CPU burst, the lower the priority, and vice versa.
- Two types:
 - ▶ Preemptive
 - ✎ A preemptive priority scheduling algorithm will preempt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process.
 - ▶ Nonpreemptive
 - ✎ A nonpreemptive priority scheduling algorithm will simply put the new process at the head of the ready queue.

Nonpreemptive Priority Scheduling

- The process, that has highest priority, is served first.
- Once a process is selected, it runs until it blocks for I/O or some event or it terminates
- This strategy can be implemented by using sorted FIFO queue.
 - ▶ All process in a queue are sorted based on their priority with highest priority process at the front of the queue.
 - ▶ When CPU becomes free, a process from the first position in a queue is selected to run.

Example: (lower integer represents higher priority and, arrival time of all process is 0 and order of arrival is P_1, P_2, P_3, P_4, P_5)

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

- Priority scheduling Gantt Chart



- Average waiting time = 8.2

Advantages:

- ▶ Priority is considered, so critical processes can get better response

Disadvantages:

- ▶ Starvation – low priority processes may never execute

NOTE: This problem can be solve by using the technique, aging.

- ☑ Aging– gradually increase the priority of processes that wait in the system for a long time.
- ☑ For example, if priorities range from 127 (low) to 0 (high), we could periodically (say, every second) increase the priority of a waiting process by 1. Eventually, even a process with an initial priority of 127 would have the highest priority in the system and would be executed. In fact, it would take a little over 2 minutes for a priority-127 process to age to a priority-0 process.

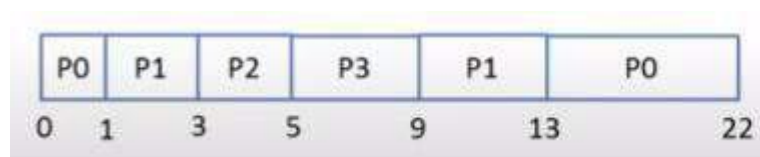
Preemptive Priority Scheduling

- The process, that has highest priority, is served first
- When a new process arrives, it's priority is compared with priority of currently running process.
 - ▶ If the new process has higher priority than the current process, the currently running process is suspended and new process is allowed to run.
- This strategy can be implemented by using sorted FIFO queue.
 - ▶ All process in a queue are sorted based on their priority with highest priority process at the front of the queue.
 - ▶ When CPU becomes free, a process from the first position in a queue is selected to run.

Example: (*lower integer represents higher priority*)

Process	Arrival Time	Burst Time	Priority
P0	0	10	5
P1	1	6	4
P2	3	2	2
P3	5	4	0

- ▶ Priority scheduling Gantt Chart



We know,

Turnaround Time = Exit – Arrival Time

Waiting Time = Trunaround Time – Burst Time

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time	Waiting Time
P0	0	10	22	$22 - 0 = 22$	$22 - 10 = 12$
P1	1	6	13	$13 - 1 = 12$	$12 - 6 = 6$
P2	3	2	5	$5 - 3 = 2$	$2 - 2 = 0$
P3	5	4	9	$9 - 5 = 4$	$4 - 4 = 0$

- Average Turnaround time = $(22 + 12 + 2 + 4) / 4 = 40 / 4 = 10$ unit
- Average waiting time = $(12 + 6 + 0 + 0) / 4 = 18 / 4 = 4.5$ unit

Advantages:

- ▶ Priority is considered, so critical processes can get better response

Disadvantages:

- ▶ Starvation – low priority processes may never execute (solution : aging)
- ▶ Context switching overhead is there.

Problem 01:

Consider the following 7 processes P1, P2, P3, P4, P5, P6 and P7. Schedule the process using nonpreemptive priority scheduling algorithm and compute the average turn-around time and average waiting time. (Lower integer represents higher priority)

Process ID	Priority	Arrival Time	Burst Time
P1	2	0	3
P2	6	2	5
P3	3	1	4
P4	5	4	2
P5	7	6	9
P6	4	5	4
P7	10	7	10

Solution:

Gantt chart:

P1	P3	P6	P4	P2	P5	P7	
0	3	7	11	13	18	27	37

Process Id	Priority	Arrival Time	Burst Time	Completion Time	Turnaround Time	Waiting Time
P1	2	0	3	3	$3 - 0 = 3$	$3 - 3 = 0$
P2	6	2	5	18	$18 - 2 = 16$	$16 - 5 = 11$
P3	3	1	4	7	$7 - 1 = 6$	$6 - 4 = 2$
P4	5	4	2	13	$13 - 4 = 9$	$9 - 2 = 7$
P5	7	6	9	27	$27 - 6 = 21$	$21 - 9 = 12$
P6	4	5	4	11	$11 - 5 = 6$	$6 - 4 = 2$
P7	10	7	10	37	$37 - 7 = 30$	$30 - 10 = 20$

- Average Turnaround time = $(3 + 16 + 6 + 9 + 21 + 6 + 30) / 7 = 91 / 7 = 13$ unit
- Average waiting time = $(0 + 11 + 2 + 7 + 12 + 2 + 20) / 7 = 54 / 7 = 7.714$ unit

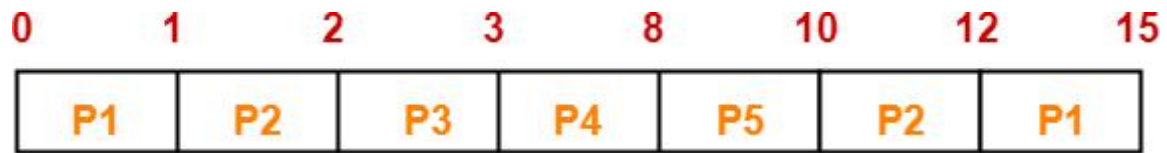
Problem-02:

Consider the set of 5 processes whose arrival time and burst time are given below. If the CPU scheduling policy is priority preemptive, calculate the average waiting time and average turnaround time. (Higher number represents higher priority)

Process Id	Arrival time	Burst time	Priority
P1	0	4	2
P2	1	3	3
P3	2	1	4
P4	3	5	5
P5	4	2	5

Solution:

Gantt Chart-



Process Id	Completion Time	Turnaround time	Waiting time
P1	15	$15 - 0 = 15$	$15 - 4 = 11$
P2	12	$12 - 1 = 11$	$11 - 3 = 8$
P3	3	$3 - 2 = 1$	$1 - 1 = 0$
P4	8	$8 - 3 = 5$	$5 - 5 = 0$
P5	10	$10 - 4 = 6$	$6 - 2 = 4$

Now,

- Average Turn Around time = $(15 + 11 + 1 + 5 + 6) / 5 = 38 / 5 = 7.6$ unit
- Average waiting time = $(11 + 8 + 0 + 0 + 4) / 5 = 23 / 5 = 4.6$ unit

Another Example:

Priority Scheduling with Round-Robin

(Smaller value of priority represents higher priority arrival time of all process is 0)

Process	Burst Time	Priority
P_1	4	3
P_2	5	2
P_3	8	2
P_4	7	1
P_5	3	3

- Run the process with the highest priority. Processes with the same priority run round-robin
- Gantt Chart with time quantum = 2

